

LLM4Rec Homework Report — Assignment A03

Collaborative Filtering Movie Recommender

Student: Chitransh Kumar Gupta **Team:** Individual **Date:** 2026-10-06 **Assignment:** A03 — Collaborative Filtering (Week 3)

Live Demo: https://chitransh-hse.github.io/collaborative_filtering/

Abstract

This assignment implements and verifies a collaborative-filtering movie recommender on the MovieLens 100K data using both user-based and item-based approaches. The implementation uses co-rated entries only for cosine similarity, selects the 20 strongest positive neighbors, and computes predictions as similarity-weighted averages. For user 1 and movie 511, the verified browser-console outputs were **4.4511** (user-based) and **4.1503** (item-based). The main takeaway is that the two methods use the same sparse rating evidence differently, so overlap requirements and sparsity are central to reliability.

Index Terms — collaborative filtering, user-based CF, item-based CF, cosine similarity, MovieLens

1. Introduction

Problem statement. The Week 3 task is to implement collaborative filtering over the supplied MovieLens files and expose both user-based and item-based recommendation/prediction behavior. The system must operate on a sparse user–item rating matrix, handle missing ratings explicitly, and provide evidence that the implementation actually runs.

Motivation. Collaborative filtering does not require movie content features: it infers preference structure from rating behavior. This makes the choice of similarity axis important. User-based CF asks which users behave similarly; item-based CF asks which movies receive similar rating patterns. Sparse data makes both questions sensitive to how missing values are treated.

Concrete example. For a selected user and target movie, the application returns two separate predictions. In the verification case user 1 / movie 511, the raw functions returned 4.4511449075 and 4.1503139596 respectively.

Contributions

- Implemented co-rated cosine similarity and both user-to-user and item-to-item prediction paths.
- Added an explicit minimum-overlap rule for target-rating prediction and verified the numerical outputs independently.

2. Related Work

Collaborative filtering was introduced as a practical information-filtering paradigm in GroupLens [2]. Sarwar et al. formalized item-based collaborative filtering, including cosine similarity and weighted prediction, and discussed performance and sparsity trade-offs [3]. Herlocker et al. emphasize that recommendation evaluation depends on the prediction task, data, and metric, so a single prediction value should not be mistaken for an accuracy benchmark [4].

Alternatives considered

- **Mean imputation:** simple and cheap, but replacing unknown preferences with averages can move similarities toward the population mean.
- **Common-count weighting:** can reduce the influence of similarities supported by very few co-rated items, at additional design complexity.
- **Matrix factorization:** a stronger model for sparse data, but outside the lightweight k-nearest-neighbor scope of this assignment.

Implementation choice

The README's default strategy was selected: **use co-rated items only**. Missing entries remain 0 in the matrix and are excluded from each cosine calculation. This avoids inventing ratings, while the target-prediction path additionally requires **20 common ratings** before a user can be considered a neighbor.

3. Method

Approach. The application loads the supplied MovieLens 100K movie metadata and ratings, constructs a $(\text{users} + 1) \times (\text{movies} + 1)$ matrix indexed by raw IDs, and represents an unrated entry as 0. The same co-rated cosine similarity is used for both collaborative-filtering variants. User-based prediction ranks similar users; item-based prediction ranks movies similar to the target among movies already rated by the active user.



Fig. 1. End-to-end pipeline used for the A03 implementation.

3.1 Sparse similarity

For vectors a and b , only positions where both ratings are non-zero are included. If the denominator is zero, similarity is defined as zero. The implementation also returns zero when fewer than five co-rated entries are available, preventing extremely small overlaps from being treated as meaningful similarity.

$$\cos(a, b) = \sum_{i \in C} a_i b_i / (\sqrt{\sum_{i \in C} a_i^2} \cdot \sqrt{\sum_{i \in C} b_i^2}), \text{ where } C \text{ is the set of co-rated positions.}$$

3.2 User-based prediction

For a target movie, candidate neighbors must share at least 20 rated movies with the active user, have positive cosine similarity, and have rated the target movie. The top 20 neighbors are then used in a similarity-weighted average:

$$r_{u,m} = \sum_{v \in N_{20}} \text{sim}(u, v) \cdot r_{v,m} / \sum_{v \in N_{20}} \text{sim}(u, v)$$

3.3 Item-based prediction

For the same active user and target movie, the implementation builds an item-rating column for the target, compares it with every other movie the user has rated, keeps the most similar positive matches, and computes the same weighted-average form using the user's observed ratings.

3.4 Tools, data, and configuration

- **Data:** MovieLens 100K; 100,000 ratings, 943 users, and 1,682 movies [5].
- **Runtime:** HTML5/CSS3 and vanilla JavaScript; no external runtime package dependency.
- **Deployment:** static GitHub Pages application; no build step.
- **Similarity:** cosine over co-rated entries; minimum 5 co-rated entries for non-zero cosine.
- **Prediction:** top-20 neighbors / similar movies; user-based target prediction additionally requires 20 common ratings.
- **Random seed:** not applicable; the implemented procedure is deterministic.
- **Hardware benchmark:** not recorded; the experiment is a functional browser verification rather than a throughput benchmark.

4. Experiments

4.1 Experimental setup

The functional experiment uses the same MovieLens files supplied with the assignment. The deployed page provides a user/movie interaction and exposes both prediction variants. Verification was performed in the browser console by calling the prediction functions directly, in addition to checking the deployed interface. Because no held-out test split or ground-truth evaluation set was recorded for this homework, the numeric predictions below are **functional outputs, not accuracy measurements**.

4.2 Verified prediction outputs

User	Target movie	User-based	Item-based	Difference
1	511	4.4511	4.1503	0.3008
10	511	4.6002	4.3497	0.2505
1	1	3.9998	4.2509	0.2511
1	50	4.7000	4.4506	0.2494

Table 1. Raw browser-console outputs used as numerical evidence. Values are shown to four decimal places; calculations used the full-precision function outputs.

4.3 User-based vs. item-based: quality and efficiency

Quality. The two predictions are often close but are not interchangeable. User-based CF transfers evidence from people with similar rating behavior, while item-based CF transfers evidence from movies with similar rating patterns. The observed numerical difference is not enough to declare one method more accurate; that would require held-out ratings and an evaluation metric such as MAE or RMSE [4].

Efficiency. MovieLens 100K has 943 users and 1,682 movies [5]. A naive full user-user similarity table contains 444,153 unordered pairs, whereas a full item-item table contains 1,413,721 pairs. Thus, full offline item-item precomputation is larger for this dataset. At target-prediction time, however, item-based CF can work from only the active user's rated movies, while user-based CF scans the user population. The practical winner therefore depends on sparsity, profile length, and whether similarities are precomputed.

4.4 Missing-value strategy trade-off

Strategy	Simplicity	Bias / risk	Cost
Co-rated only	High	Can be unstable with few overlaps	Low
Mean imputation	High	Can pull unknowns toward the mean	Low-medium
Common-count weighting	Medium	Less trust in weak overlaps	Medium
Matrix factorization	Lower	Model-dependent; learned assumptions	Higher

Table 2. Qualitative trade-off analysis; no unrecorded benchmark is claimed.

4.5 Manual verification

A unit-level cosine calculation was performed independently to verify the implementation formula. For $a = [5, 4, 3, 2, 5]$ and $b = [4, 5, 2, 3, 4]$, all five positions are co-rated: $\text{dot}(a, b) = 72$, $\|a\|^2 = 79$, $\|b\|^2 = 70$, giving $72/(\sqrt{79} \cdot \sqrt{70}) = \mathbf{0.96821}$. This is a formula-level sanity check, not a claim about a MovieLens user pair.

5. Discussion

5.1 Real failure case and root cause

Observed failure

ReferenceError: MIN_COMMON_ITEMS is not defined was observed when the prediction function was invoked from the browser console before the constant was available in the loaded execution scope. The prediction function depends on that constant when filtering candidate users.

Root cause. JavaScript const bindings are not usable before initialization. The console call therefore failed before the prediction computation itself could run. This was a runtime/scope issue rather than a mathematical error in cosine similarity.

Fix + verification. The constant was placed in the loaded script before the function was exercised, and the prediction path was re-run. The subsequent direct calls returned valid numeric results, including user 1 / movie 511 = 4.4511449075 (user-based) and 4.1503139596 (item-based).

5.2 What worked

- The sparse matrix convention 0 = not rated makes missing-value handling explicit and compatible with MovieLens ratings 1–5.
- The 20-common-rating gate prevents a user with only a tiny overlap from entering the target prediction pool.
- Direct function calls made it possible to inspect raw numerical outputs rather than relying only on formatted UI values.

5.3 What surprised me / did not work

- The two CF variants can produce noticeably different predictions even when they use the same cosine similarity formula, because their evidence comes from different axes of the matrix.
- Very sparse overlaps make cosine similarity fragile: a high similarity supported by only a few shared ratings can look stronger than it deserves, which motivated the overlap thresholds.
- A prediction value by itself does not establish recommendation quality. Without held-out ground truth, choosing the larger of two predictions would be unjustified.

5.4 Cold start and sparsity

Collaborative filtering cannot infer a meaningful neighborhood for a completely new user with no ratings, because there is no behavioral vector to compare. The same issue appears for a new movie with no ratings: item-item similarity has no co-rated evidence. Even when ratings exist, very few common items make similarity statistically fragile. The minimum-overlap rule is therefore a reliability safeguard, not a solution to cold start.

5.5 Top-Down 5-Loop trace

Loop	Action in this assignment	Evidence
BUILD	Ran the deployed recommender and prediction functions.	Working UI + console outputs.
ASK WHY	Questioned co-rated-only handling, the 20-overlap rule, and user-vs-item axis.	Alternatives and trade-offs in §§2 and 4.
TEST	Checked raw outputs and independently computed one cosine value.	Table 1 + manual 0.96821 check.
EXPLAIN	Interpreted the two prediction paths in terms of matrix axes and sparsity.	§§3–5 discussion.
IMPROVE	Added explicit target prediction paths, overlap filtering, and verification calls.	Re-run produced valid numerical outputs.

Table 3. Evidence trace aligned with the course's required BUILD → ASK WHY → TEST → EXPLAIN → IMPROVE loop.

5.6 Next improvement

With more time, I would add a reproducible train/test split and report MAE/RMSE for both methods, followed by a controlled comparison of the minimum-overlap threshold. That would turn the present functional comparison into a measurable recommendation-quality study while preserving the same implementation.

6. AI Usage Disclosure

The course requires an honest record of how AI contributed to the work. ChatGPT was used during implementation and report preparation; the numerical claims below are limited to outputs that were checked against the code and browser behavior.

AI tool and use

AI tool	Use	Personal verification
ChatGPT (GPT-5.6 Luna)	Code implementation/debugging, algorithm explanation, report organization, script logical review	Inspected organization, script logical review, browser-console prediction functions; checked

Table 4. AI-use disclosure and verification evidence.

What I personally verified

- The prediction functions were called directly in the browser console and returned numeric values rather than errors after the scope issue was fixed.
- The target prediction for user 1 / movie 511 was independently recorded as 4.4511449075 (user-based) and 4.1503139596 (item-based).
- The cosine calculation was checked manually with a five-overlap vector example, producing 0.96821.
- The MovieLens dimensions used in the report (943 users, 1,682 movies, 100,000 ratings) were cross-checked against the published dataset description [5].
- No held-out accuracy benchmark was invented; the report explicitly labels the prediction values as functional outputs.

What I trusted without independent verification

The report-writing structure, wording suggestions, and some implementation explanations generated by AI were treated as drafting assistance and reviewed for consistency. I did not treat an AI-generated numerical claim as evidence unless it was re-run or manually checked.

Session log

Submission note

The official guidelines require a separate, unmodified tool-native **session.json** audit trail. This PDF does not fabricate or rename a ChatGPT conversation as that file. If a qualifying tool-native session log exists from the coding workflow, it should be submitted separately using the course naming convention.

Conclusion

The A03 implementation provides both user-based and item-based collaborative-filtering predictions over the MovieLens 100K sparse rating matrix. The chosen co-rated-only similarity is simple and avoids inventing unknown ratings, while the 20-common-rating gate reduces the risk of relying on extremely weak overlaps. Verification combined direct browser execution, a real runtime failure-and-fix trace, and a manual cosine calculation. The main remaining limitation is evaluation: without a held-out test protocol, the two methods can be compared mechanistically and operationally, but not honestly ranked by predictive accuracy.

Live application

https://chitransh-hse.github.io/collaborative_filtering/ — deployed static demo used for functional verification.

References

The following references were selected to support the collaborative-filtering formulation, item-based CF, evaluation considerations, and the MovieLens dataset used in the experiment.

- [1] S. Jin, *LLM4Rec — Homework Guidelines*, HSE University, official student-facing guidelines, 2026.
- [2] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "GroupLens: An Open Architecture for Collaborative Filtering of Netnews," in *Proc. ACM Conf. Computer Supported Cooperative Work (CSCW)*, pp. 175–186, 1994. doi: 10.1145/192844.192905.
- [3] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proc. 10th Int. World Wide Web Conf. (WWW)*, pp. 285–295, 2001. doi: 10.1145/371920.372071.
- [4] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, "Evaluating collaborative filtering recommender systems," *ACM Trans. Information Systems*, vol. 22, no. 1, pp. 5–53, 2004. doi: 10.1145/963770.963772.
- [5] GroupLens Research, "MovieLens 100K Dataset," University of Minnesota. Available: <https://grouplens.org/datasets/movielens/100k/> . Accessed: 2026-10-06.

Evidence note

The report intentionally distinguishes source-backed dataset facts and algorithmic claims from functional outputs observed in the student's browser. Prediction values are reported from direct function calls and are not presented as benchmark accuracy. This avoids a claim–evidence mismatch under the course's Gate 0 rule.

Submission files: Chitransh_Kumar_Gupta_A03_report.pdf and the separate required Chitransh_Kumar_Gupta_A03_session.json if a qualifying tool-native session log is available.